STM JOURNALS

Journal of VLSI Design Tools & Technology
ISSN: 2249-474X (Online), ISSN: 2321-6492 (Print)
Volume 10, Issue 3
www.stmjournals.com

Hierarchical Methodology Approach to SOC Design: A Comprehensive Look

Vivek Bhardwaj*
Independent Researcher, CAD division, Intel Corporation, Penang, Malaysia

Abstract

The design scale of a chip has increased many folds in the last few years and shows a continuous exponential trend. This is made possible by the reduction in transistor size and an evolving process technology. As a result, it is possible to move to SoC technology (System on Chip-A single die consisting of multiple subsystems) rather than a traditional ASIC. This has obviously resulted in an increase in the instance/gate count and the functionality of the chip resulting in a multi core subsystems based designs. Hence software scalability is needed to support the increasing complexity in the function of the chip and the related timing analysis and operations based on it. Originally for small ASICS flat implementation was sufficient and was most accurate. For SoC's it is replaced with a modular approach in handling designs which is now is further improved for accuracy using the concept of Interface Models that have various degrees of edit ability and accompanying flows. This paper starts with flat description 'of flow implementation traditionally used by EDA tool companies and then go on to comprehensively describes hierarchical implementation flows and their artifacts like, models, post assembly ECO, context views, feasibility flows among others. We are also going to look at advances in the tool technology of timing graph reduction and physical data reduction that enables new flow implementations and can be applied to both hierarchical and flat methodologies.

Keywords*: ASIC, ECO, EDA, interface models, SoC technology*

****Author for Correspondence*** E-mail: Vivek.bhardwaj@intel.com

INTRODUCTION (PROBLEM STATEMENT)

There is an ever-increasing need for the software to handle the large gate count with faster turnaround time and better accuracy [1]. Figure 1 shows the trend approximately. Reaching timing closure on multi-million gate VLSI chips using flat flow is infeasible due to hardware capacity limit, and excessive run time overheads. Over many years EDA (Electronic design automation) companies have devised new ways and automation to solve this problem. Sometimes they have taken inputs from the design companies and sometimes they have come up with innovative solutions to handle A typical analysis of designs shows that the memory requirements of the STA database (Static Timing Analysis) is much larger than the physical database. This is predominantly due to a large increase in number and complexity of clock functions and the timing exceptions and constraints spanning the entire chip. As a result, scaling down the memory requirements of the timing analyzer is critical to contain the overall memory reduction of the entire implementation. Similarly, the runtime of the entire chip implementation also increases with the memory since the data structure size increase and their order of complexity also increase, thereby proportionately increasing the algorithmic complexity of the entire software handling of the SOC**.** From timing closure perspective, it can be attributed to an increased number of critical paths (that need closure) and the overall complexity of timing analysis that just not impacts the STA but all flow applications that uses STA, e.g. timing driven place and route [2], physical optimization [3], timing budgeting [7] clock tree synthesis [4], etc. It has been observed that timing aware placement leads to an exponential jump in run time as compared to non-timing aware placement when the design size reaches in excess of 10 million gates.

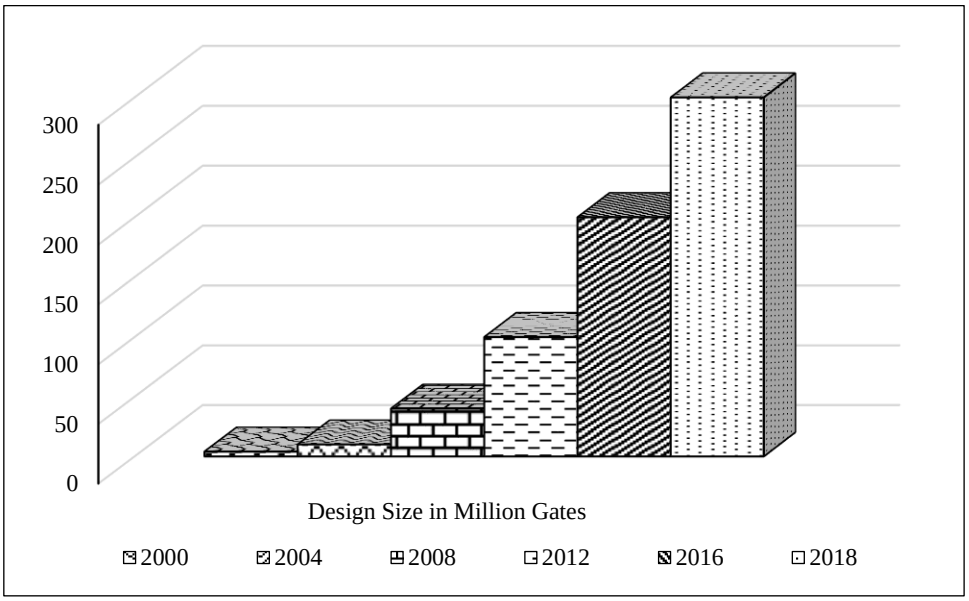

Fig. 1: *Design scalability trends (Moore's law and new nodes).*

Hence, to manage the desired level of scalability, appropriate design implementation strategy is to be adopted. A full flat implementation offers maximum accuracy and simplicity as opposed to traditional hierarchical implementation that offers maximum scalability and is quite a complex flow compared to flat. The hierarchical implementation employs the use of the black box timing models and Interface Models to represent portions of the design (referred to as partitions/blocks/subsystems/sub chips in this paper). when processing large chips Somewhere, in between these implementations and used at various stages are the various context abstractions and design abstraction techniques that are employed to simplify a huge design for point operations and design planning.

In the next section, we are going to describe the traditional full flat and hierarchical flow in detail and look at their key steps and understand their advantages and drawbacks and typical usage. We are going to look at the Interface Models of various degrees of freedom and the way it improves the traditional hierarchical flow. We will also overview some non-traditional and out of the box approaches like context views and prototyping techniques and understand the technology usage. In the last section we are going to provide a conclusion on typical usage and pros/cons.

TRADITIONAL IMPLEMENTATIONS

Flat Implementation

The flat implementation of the chip is the simplest methodology that offers the fastest turnaround time solution with respect to timing closure for smaller sized designs. The biggest virtue of flat implementation is that the entire design is visible to the software tool and a unified constraint file is developed and used for the full SoC including its sub chips (also referred as sub systems) and the top level logic. The flat approach enables a seamless view of the subsystems and all the nested hierarchies. From EDA software point of view, the entire design collateral (SDC, UPF, RTL, switching activity etc.), is loaded (called as reading in the files) in software's internal data structures that occupy memory in the OS/computer hardware. When implementing such a design, there are no back and forth iterations involved between top level and the designs' sub-systems and hence it is a one shot implementation. The obvious drawback of such an approach is scalability since the run time and memory usage increases drastically. This flow is fit for design with 1–2 million placeable instances. Depicted below in Figure 2 is a typical flat flow.

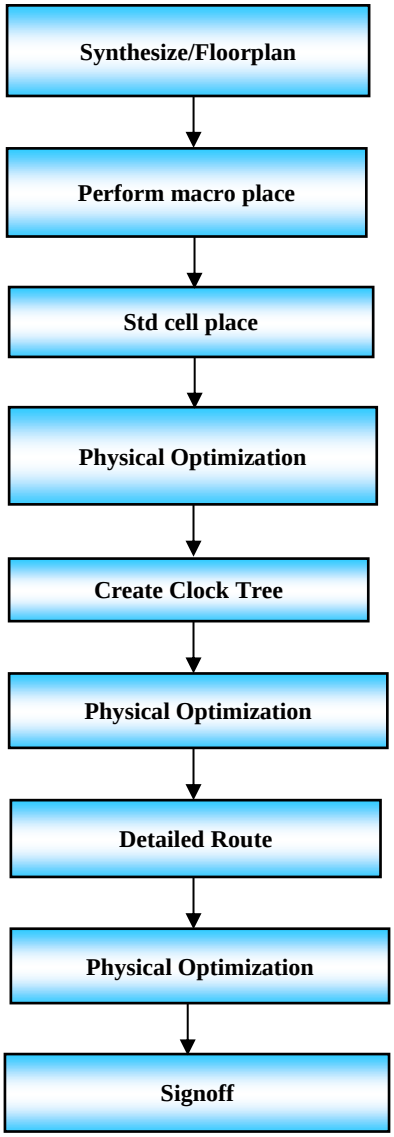

Fig. 2: *Flat Implementation Flow.*

As can be seen from the above diagram that optimization at each step in the flow is done for the entire flat chip including the logic within the sub-systems. There are indeed innovations done at the flat level as well but at some point, a hierarchical approach becomes a necessity.

Hierarchical Implementation

To resolve the problem of scalability for handling large designs, hierarchical solution was developed progressively by the combined thought of SoC designers and EDA developers. It enables breaking down of entire design into individual design components which is process called as partitioning the design, and then handling each individual design in a flat manner. It is essentially a divide and conquer approach that lets designers strategize their SoC implementation in either a top down manner or a bottom up implementation [5, 6]. In the typical top down hierarchical flow. the design is partitioned into sub designs and each component is implemented individually and in parallel. The essential part of top down implementation is planning the top level collaterals in such a way that they at least cover top level logic and the interface logic at a minimum. This is such that the interface logic and generated physical database can be pushed down to the sub blocks. Once each sub-design/block is implemented, then a model is created out of that block. The models are then assembled back in the top level for the final top implementation. The hierarchical flow approach can be used during the prototyping and implementation phases. During the prototyping phase (also called as early feasibility stage), the chip netlist is not complete. For the incomplete/empty Verilog modules, black boxes could be created and initial STA could be done using black box what-if timing analysis. The estimation results reflect the state of the design it is in. Hence hierarchical approach becomes a must here. One can argue here that we can use incomplete netlists and do a feasibility check, however that adds to the design memory/runtime which is not essential and lead to interface paths that are sub-optimal. Later, in the implementation phase, physical partitioning is done which includes global routing, pin assignment and time budgeting followed by saving the partitions as individual sub designs.

The time budgeting step produces timing constraint file for each partition which can be used for block implementation [5, 6]. Various unique flows are devised by the software to represent the budgets in a way that the timing requirements of data and clocks are met [8]. The software also provides visual and textual ways to depict the budgets to the designer. Budgeting also generates a top level constraint file and a timing library representation of each partition. These outputs are relevant in simultaneous top level implementation along with the blocks. A word of caution about time budgeting is that budgeting does not create

time. It only allocates budgets (or slacks) to the internal and external portion of the boundary paths such that they represent the path slacks in a proportionate manner at the top level and the partition level. If a path is positive slacked, then excess positive budget is redistributed (often referred as margin) and reversely if a path is negative slacked, then the negative slack is redistributed. Modern EDA tools provide software mechanism to shift these slacks from one side to another which is a way to provide the designers with a flexibility to customize their budgets in the implementation stage itself. Often the designers know their designs and how the design implementation is to be done at a later stage, which is why they can choose to shift the budgets to either side of the boundaries, but that is not the default way. As said earlier, the budgeting process does not create time and merely does an automation of the proportion allocation to external and internal. Hence, in order to model something that is going to happen later in the flow, some tools do some kind of virtual optimization (that is not physical or real) on the negative slack paths to arrive at a more realistic budget. One can argue that why not do a full physical optimization of the design and not just virtual but doing so results in lot of runtime and compute and defeats the purpose of hierarchical implementation. However, there are some alternative flows that will be described later that can reduce the scope (or context) of the design and hence lead to lesser memory/runtime. Also, there is a concept of re-budgeting that some designers adopt where they do refined budgeting based on one round of physical optimization that has already happened. To conclude about budgeting, there are two rules to follow-firstly your output is as good as your input and secondly each refinement or accuracy consumes overall runtime by adding a flow stage. So, accuracy comes at cost of runtime.

Another key step in partitioning the design is interface pin assignment at the boundaries of the hierarchical block. Accurate pin assignment [5, 6] is a critical component of the flow where interface pins are allocated a definite physical location on the boundaries. These locations are critical as these are the entry points to a block or a partition. The physical wires cross through these locations and hence the locations indirectly determine the timing of a critical path that crosses the boundary. The timing of a path is dependent on the wire loads which are in turn is dependent on the routing topology [5, 6] that wires used by the timing path undertakes to reach its destination. Hence determination of ideal or close to ideal pin locations are key. The location also becomes important to minimize the pin access issues that a router must resolve in order to be design rule aware [5, 6]. The metal layers used by the pins also become important in deciding the opens/shorts faced later on during final implementation. In order to arrive at a close-to-ideal locations, the software methodology does some kind of rough/fast routing to give approximate routing guidance to the pin assignment flow. The routing in turn is timing aware. The pin assigner takes this as guidance but does not adhere fully due to physical rule compliance as mentioned above. Hence a second pass of routing is advisable before timing budgeting and after pin assignment so that budgets can be calculated based on final routes that cross through the pin locations.

Third step is the physical step of partitioning the design which is a mechanical and software centric step of deriving new databases for each subchip and the top. This step is essentially transparent to the designer and ensures that each database has its own sets of collaterals (netlist, UPF, SDC, floorplan) to implement a block independently. Block implementation is similar to flat process but happens on much smaller scale. There are some flows that do hierarchical implementation inside a sub-system as well. These are called as nested partitions. The goal of block implementation is timing/area/power closed block.

Top Level Implementation

After each individual block is implemented (post Route optimized timing closed), a model is generated that is assembled back in top level. This model could be black box timing model or an interface model. We are going to see about interface models in detail in next section. In the top implementation, the block is a black box (empty Verilog) but has a timing representation as. lib model [5]. Top level implementation is sometimes used

progressively by designers where latest data from blocks is being refreshed and fed to the top level implementation lead. This data refresh is based on either availability of latest collaterals or availability of more recent flow step (like post clock tree model replacing the pre clock tree model or post detailed route model replacing the post clock tree model) or is a combination of both. The goal of top implementation is timing/area/power closed top with some form of hierarchical representation of blocks.

Assembly

Assembling the design is the last hierarchical step in the flow where top level database and block level databases are stitched together to complete or flatten the SOC once again. At assembly stage, one can expect the design to be closed just like it was at individual block and top stage. However, in reality that seldom happens since there are lot of factors that affect the final result. This happens due to multiple factors like hierarchical pushdowns of imperfect timing budgets, routes, incorrect pins assigned, incorrect models generated for blocks, side loads missing in a view, signal integrity issues that can't be perfectly modeled, etc. At this stage, re-budgeting is performed to generate new budgets and iterate the process. Tools have advanced flows for closure after assembly that we will touch down on that later.

There are several advantages of hierarchical flow. Firstly, it is the only scalable flow that can close very large designs. For design sizes with more that 5–7 million placeable instances, in present context, only hierarchical flow can be used to scale down the turnaround time and the memory usage. Also, as we have seen already, parallel implementation of blocks and top along with initial prototyping with black boxes offer a big advantage.

However, traditional hierarchical implementation has some inherent drawbacks, simply attributed in the way things are. It relies on timing model generation to do top level implementation. The model generation, if inaccurate, can lead to incorrect timing analysis results. Also, since the complete path is not visible (since the partition is a black box), the external environment effects like slew propagation may be incorrectly accounted for in delay calculation leading to incorrect timing. Also, the accuracy of timing analysis depends on budgeter's ability to derive top level constraints and exceptions used during top level implementation. Such drawbacks can increase the number of iterations needed to close design timing. Some of the drawbacks could be managed by using a better design style where the interface logic and exceptions are kept to a minimum to reduce chances of inaccuracies. These drawbacks can also be resolved by modifying the traditional hierarchical flow to use interface models instead of black box timing models. We will study this in more detail next.

Another issue is the post Route timing closure. Post Route is the last step in physical design (also called as construction or structural design) and closing timing. Since black boxes or interface models are not allowed to optimize the logic of the timing path segment inside the partition block, any violation on the top level that cannot be fixed on top level itself needs to be fed back to the block through ECO or even re-budgeting and the block might need re-implementation. Besides this problem of several passes, any hierarchical implementation relies on tool's ability to abstract the design (interface models or black boxes).

New Trends

Interface Models

Interface Models are advanced versions of black boxes that have completely empty netlist. It is a reduced view of a block that contains physical and logical data which is relevant only to the block interface. Here the original circuit is modeled by a smaller circuit that contains the logic leading from the input ports to interface sequential elements and from interface sequential elements to output ports. This enables interface timing paths to be preserved. Such models are composed of the truncated netlist, parasitic RC info, timing constraints, physical placement info and if required, aggressor nets info that can be used for top level SI fixing. The internal register to register path within the block is not exposed in the model as these paths are not affected by the budgeted constraints and are fixed in the block implementation phase itself. Such

models can be used with various timing driven applications like timer, optimizer, router etc. to do a top level implementation (Figure 3).

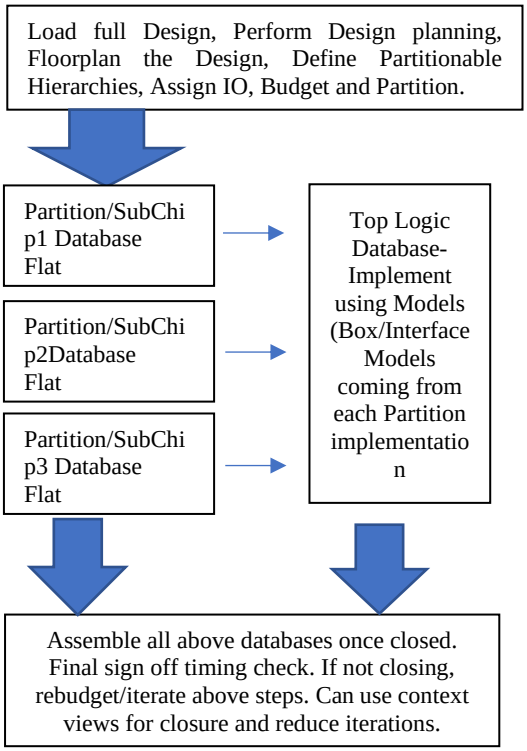

Fig. 3: *Hierarchical Top Down Flow.*

In Figure 4., the circles depict partitions or sub chips. The interface modes create a donut type of shapes where the solid portion (shown in green) is visible along with the top while the hollow portion (shown in black) is removed for both timing and physical database. This whole depiction is figurative whereas in practice there are models dumped out that have hollowed netlists and the associated parasitic with it.

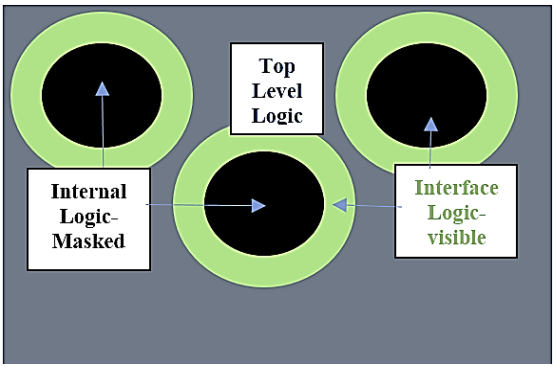

Fig. 4: *Interface Mode of a Chip.*

These models eliminate the drawbacks of the black box hierarchical flow as listed in the previous section. Since the entire path segment inside the block is available at top level, timing analysis accuracy is much better. But like everything, there is a penalty on runtime, slight memory increase (because of interface) and flow complexity.

However, there is still one drawback. Such models can't be optimized or edited. Because of the way optimization transforms work, sometime an interface path can only be fixed if fixed inside a block. Hence, some EDA tools have also the concept of editability of such models. To be able to be editable, these models have to bring their physical properties like routing and placement. Hence if a model has to be able to be optimized, then it has to follow a different flow. Again, this increases the complexity and memory requirements incrementally over the last improvement.

Contextual Design Views

Certain EDA tools provide capabilities to see the design in a narrower/subset view of the full design. This is done to enable the memory/runtime footprint but at the same time have the ability to see the portion of the design netlist that is relevant to do a certain task. E.g. doing timing on interface paths only or optimizing only critical paths or doing full physical optimization in order to determine timing budgets or pin location refinement or even closing the design after assembling the chip in full post route mode. The purpose of contextual views is to focus the timing analysis on a part of netlist in order to reduce run time and memory. The timing view could be applicable to any timing related process like placement, optimization, timing analysis, etc. Once the portion of the design is masked, the application, such as timer or optimizer sees only a small timing database related to small portion of the design and can work on it using much smaller memory footprint. As a result, run time also improves. Specific examples if contextual views are top +interface which is similar to Interface model view and critical nets only that span in and out of the hierarchies regardless of any boundaries. A major advantage of such technology is that relevant portion is available when required and multiple such sessions could be created on

demand for various purposes like optimizing for signal integrity in parallel with power optimization. However, any such technology is different from the true hierarchical approach of using models for the sub systems or partitions. Careful analysis needs to be done whether such features do not violate the hierarchical structure of routing while doing physical optimization or needless tempering of entry ports to the sub chips.

Feasibility Flows

A SOC designer or an integration architect for the full chip needs to sometimes see compressed or abstracted views of the full netlist (and not just interface or top or subchip-only) in order to make design decisions early in the cycle. E.g. a designer might need to make floor planning or local density decisions based on estimated routing congestion in a specific region of the chip that has critical logic going through a block or inside a block. Another example could be designer has to perform some kind of what if analysis where a block is expected to have certain kind of timing and load and drive strength and top level architect needs to do some kind of plug and play analysis to arrive at correct functional logic for the top level-the underlying assumption here is that the subsystem or partition designer has to perform or close the blocks in a certain budget-whether it is timing or area or power. To do these kinds of fast analysis and accompanying slack/power calculations, modern EDA tools provide extensive design planning or what they call as prototyping tools. Such tools provide extensive artifacts and analysis techniques to enable designers or architects to perform such analysis based on certain core assumptions that all modern day SOC's and power/area/performance hungry chips have to adhere to.

CONCLUSIONS/SUMMARY

In this paper we outlined various techniques and flows available today in modern EDA tools to accomplish every challenging task of taking a multi-million (running in tens of millions) instance/gate count design to completion in a timely manner. This challenge is primarily due to shrinking transistor geometry and the ever-increasing consumer expectations leading to innovation both in CAD/EDA methodology as well as design methodology. There are various capabilities/sub-capabilities that exist today ranging from flat, context driven flat/hierarchical, model based hierarchical approaches and early feasibility flows. Some of these techniques are replaceable based on design size and flow complexity desired. However, some of them are non-replaceable if huge design sizes are to be achieved in limited schedule time and with limited computer hardware resources. When choosing a hierarchical methodology, meticulous planning needs to be done from the start of the design like number of partitions, size and shape of each partition, bus and interconnect planning, power structure and low power hierarchical considerations, top level area overheads due to hierarchical constraints, Some techniques are alternate to each other and are to be employed by designers knowing their design situation. Current large SOC tapein/tapeout schedules range from six months to one year and it is observed that a vast amount of the time goes in chip planning which is to be done for various collaterals refresh that happen in the entire design cycle. This is where the design planning tools come into picture and become practically irreplaceable. Also, for medium sized designs where intent is to enable parallel distribution the hierarchical approach is a must. So, in summary, there are various tools available to a SOC designer/integration lead in today's rapidly increasing automation of electronic chip design that the designer can study and adapt to her/his design needs, to attain an implementable project schedule.

REFERENCES

1. Bhardwaj, Vivek, Didier Seropian and Oleg Levitsky. Methods for generating a user interface for timing budget analysis of integrated circuit designs. United States of America: Patent US8745560B1. 3 June 2014. U.S. Patent and Trademark Office. <https://patents.google.com/patent/US8745560B1/en>.
2. Bhardwaj, Vivek, Didier Seropian and Oleg Levitsky. User interface for timing

budget analysis of integrated circuit designs. United States of America: Patent US8504978B1. 6 August 2013. <https://patents.google.com/patent/US8504978B1/en>.

3. Bhardwaj, Vivek, Oleg Levitsky and Dinesh Gupta. Flow methodology for single pass parallel hierarchical timing closure of integrated circuit designs. United States of America: Patent US8365113B1. 29 January 2013. <https://patents.google.com/patent/US8365113B1/en>.
4. Bhardwaj, Vivek, Oleg Levitsky and Dinesh Gupta. Machine readable products for single pass parallel hierarchical timing closure of integrated circuit designs. United States of America: Patent US9165098B1. 20 October 2015. <https://patents.google.com/patent/US9165098B1/en>.
5. Bhardwaj, Vivek, Oleg Levitsky and Dinesh Gupta. Methods for single pass parallel hierarchical timing closure of integrated circuit designs. United States of America: Patent US8935642B1. 13 January 2015. <https://patents.google.com/patent/US8935642B1/en>.
6. Bhardwaj, Vivek, Oleg Levitsky and Dinesh Gupta. Systems for single pass parallel hierarchical timing closure of integrated circuit designs. United States of America: Patent US8539402B1. 17 September 2013. <https://patents.google.com/patent/US8539402B1/en>.
7. Dinesh Gupta, Oleg Levitsky. Multi-phase models for timing closure of integrated circuit designs. United States of America: Patent US8640066B1. 2014.
8. Sumit Arora, Amit Kumar. Timing budgeting of nested partitions for hierarchical integrated circuit designs. United States of America: Patent US8977995B1. 2015.

Cite this Article

Vivek Bhardwaj. Hierarchical Methodology Approach to SOC Design-A Comprehensive Look. *Journal of VLSI Design Tools & Technology*. 2020; 10(3): 35–42p.

